Final Project Report

Design and Implementation of an 8-bit Mini CPU with Stack

(Mini CPU + Stack)

Course 2110252 Digital Computer Logic
CEDT Final Project 2569 (2026)

Prepared by Team "จอมมารโง่"

6933076221 Natthatida Hutangkoon
6933277021 Sukrit Tongsri
6933212221 Phuripong Deeted
6933064721 Chanawat Chobtumkit

Submitted to the Instructor of 2110252 Digital Computer Logic

Department of Computer Engineering, Faculty of Engineering
Chulalongkorn University

Academic Year 2026 (B.E. 2569)

Chapter 1 Introduction

This project is part of the course 2110252 Digital Computer Logic. The goal is to design an 8-bit CPU using the Digital simulator. The CPU receives a program into a program RAM (pRAM) of 256 × 14 bits and executes instructions from address 0x00 until it reaches a STOP instruction. It then waits for a signal before sending out the contents of the result RAM (rRAM, 256 × 8 bits), locations 0x00–0x0F, through the output port and two 7-segment displays. A stack is located in the upper part of rRAM. The objective is to design a Datapath and a Control Unit (an 8-state FSM) that support the 42 basic instructions, additional instructions placed in reserved opcode slots, and hardware loops for GCD, FIB and SSUM.

Table 1 Specifications and signals of the Mini CPU

Item	Description
Data and instructions	8-bit data; 14-bit instructions (6-bit opcode [13:8] and 8-bit operand [7:0])
Registers	accA, accB (accumulators), regC, regD and SP (8 bits each), plus PC (8 bits) and IR (14 bits)
Flags	eq, gr, ls (1 bit each), used by conditional jump instructions
Inputs	M, N (external values), progIn (14-bit instruction), progLoad (write to pRAM), start, result, reset, clk
Outputs	valid (STOP reached), done (all 16 results sent), output (8 bits) and the 7-segment displays

Note: an accumulator is a register that holds the value fed into the ALU and receives the result back.

Chapter 2 Architecture and Instruction Set

2.1 Memory and Stack

pRAM stores the program, while rRAM stores data and results. Locations 0x00–0x0F are the result area that is sent out at the end of execution. The FIB result is stored at 0x0A (high byte) and 0x0B (low byte), and the SSUM result at 0x0C and 0x0D. The stack resides in the upper part of rRAM. According to the circuit, SP starts at 0x00 (empty stack); PUSH decrements SP by 1 and then writes the data, while POP reads the data at SP and then increments SP by 1. The flags are updated by CMP (compares accA with accB) and isPrime (checks whether accA is prime using a Prime ROM and reports the result in the eq flag).

2.2 Instruction Format and Instruction Groups

opcode[5:4] selects the instruction group (00 Data, 01 Memory/Stack, 10 ALU, 11 Control/Special), opcode[3:2] selects the sub-group, and opcode[1:0] selects the instruction within the sub-group (up to 4). Unused slots are reserved for the special (Secret) instructions. The basic set has 42 instructions (Data 12, Memory/Stack 7, ALU 13, Control 10), and three additional instructions (CALL, RET, SSUM) are placed in reserved slots.

Table 2 Instruction groups and opcode ranges

Group	Opcode	Sub-groups (number of instructions)
00 Data	0x00–0x0F	Immediate: load an 8-bit constant into accA, accB, regC (3) | Copy from accA/accB (4) Copy into accA (3) | Read inputs M, N (2)
01 Memory / Stack	0x10–0x1F	Direct memory: read/write rRAM (3) | Indirect memory via pointers regC/regD (2) Stack: PUSH, POP (2) | Subroutine: CALL, RET (reserved slot)
10 ALU	0x20–0x2F	ADD 0x20, SUB 0x21, MUL 0x22 | DIV 0x24, MOD 0x25 AND 0x28, OR 0x29, XOR 0x2A, NOT 0x2B | SHL 0x2C, SHR 0x2D, ROL 0x2E, ROR 0x2F
11 Control / Special	0x30–0x3F	Branch: JMP 0x30, JEQ 0x31, JGR 0x32, JLS 0x33 | Loop: DJNZ 0x34 CMP 0x38, isPrime 0x39 | GCD 0x3C, FIB 0x3D, SSUM 0x3E (Secret), STOP 0x3F

2.3 Special Instructions

- CALL / RET: CALL stores Next_PC on the stack and jumps to the address in the operand; RET reads the stored value from the stack back into PC.
- GCD: copies accA and accB into gcdA and gcdB, then subtracts repeatedly (if A > B then A ← A − B, otherwise B ← B − A) until A = B, and returns the result to accA.
- FIB: uses accA[4:0] as the iteration count fibN, starting with fib0 = 0 and fib1 = 1. Each iteration performs fib0 ← fib1, fib1 ← fib0 + fib1 until fibN = 0, then writes the 16-bit fib0 to rRAM 0x0A–0x0B.
- SSUM: sets ssumPtr ← SP and ssumAcc ← 0 (an empty stack is detected by ssumDone), adds the stack data up to address 0xFF, and writes the 16-bit sum to rRAM 0x0C–0x0D.

Chapter 3 Design Concept

The design is divided into four steps. The Datapath (computation and data storage) is kept separate from the Control Unit (timing and control signals), so a new instruction can be added by modifying the Decoder ROM and the related data paths.

Table 3 Design steps

Step	Tasks	Goal
1 Specification and instruction format	Define an 8-bit data bus and 14-bit instructions, allocate 42 opcodes into 4 groups, and divide memory into the pRAM and rRAM zones.	A complete instruction-set map

Step	Tasks	Goal
2 Datapath	Build the ALU modules (Add, Sub, Mul, Div, Logic), connect registers accA/B, regC/D, SP, PC, and build the data bus with MUXes.	Computation circuits work correctly
3 Control Unit	Design the FSM (states 000–111), add a Decoder ROM that converts opcodes into control signals, and wire the clock-enable and MUX signals.	Basic instructions and jumps run
4 Hardware loops and testing	Connect the dedicated GCD, FIB and isPrime circuits and the display system (rRAM to 7-segment), then test the whole system.	Pass the test cases, including Secret instructions

Chapter 4 ASM Chart and Control Unit

The Control Unit is an FSM with 8 states, stored in a 3-bit register, as shown in the ASM chart in Figure 1.

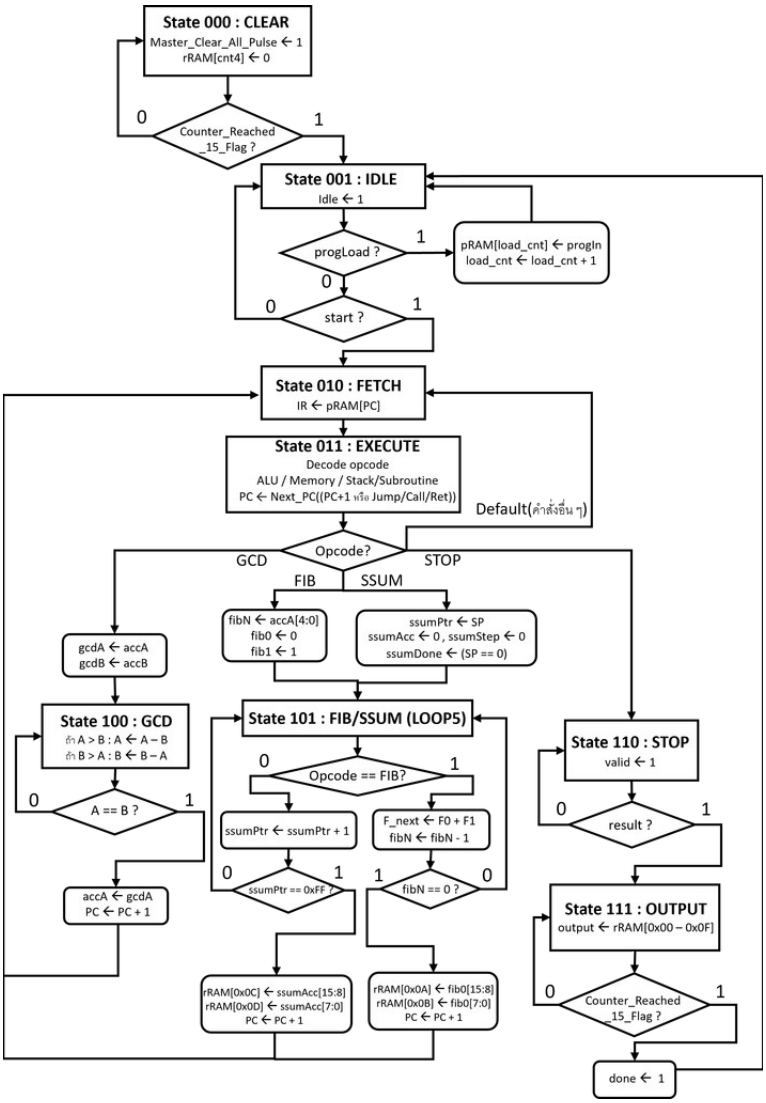


Figure 1 ASM chart of the Mini CPU

Table 4 FSM states

State	Operation
000 CLEAR	Sends the Master Clear pulse to clear registers and flags, and writes 0 to rRAM one location at a time using cnt4 until all 16 locations are cleared, then moves to IDLE.

State	Operation
001 IDLE	Waits for the user. If progLoad = 1, writes progIn to pRAM[load_cnt] and increments load_cnt. If start = 1, resets PC and SP and moves to FETCH.
010 FETCH	Reads pRAM[PC] into IR.
011 EXECUTE	Decodes and executes the instruction (ALU, Memory, Stack, Subroutine) and computes Next PC (PC + 1, jump, CALL, RET), then branches to GCD, FIB/SSUM, STOP, or back to FETCH.
100 GCD	Subtracts A and B repeatedly until A = B, moves the result to accA, then PC + 1 and returns to FETCH.
101 FIB / SSUM	Loops according to the opcode; when finished, writes the result to rRAM, PC + 1 and returns to FETCH.
110 STOP	Sets valid = 1 and waits for result = 1, then moves to OUTPUT.
111 OUTPUT	Sends rRAM[0x00–0x0F] to the output and 7-segment displays over 16 cycles, sets done = 1 and returns to STOP.

Chapter 5 Circuit Design in Digital

The circuit is organized into sub-circuits by function (blue = Datapath, red = Control Unit). The key points are as follows.

- Input/RAM/Output/Decoder ROM (Figure 2): the Decoder ROM converts the 6-bit opcode into control signals such as Instruction_DemandsWrite A–D (request to write a register), Select_InputSourceFor A–D (MUX source selection), Memory_WriteRequest, Select_rRAMAddressMode and Stack_OperationActive.
- Registers & State Decoding (Figure 3): the STATE register, counters cnt4 and load_cnt, PC, IR, valid, done, and comparators that decode STATE into State_IsClearAll … State_IsOutputStreaming.
- Next-State Logic (Figure 4): MUXes select the next state from the current state, together with opcode detectors for STOP, GCD, FIB, CMP, isPrime and DJNZ. The reset signal forces the FSM back to CLEAR.
- Stack, Registers & RAM Mux (Figure 5): accA can load from the operand, accB, regC, regD, SP, rRAM, pRAM or the ALU. SP selects +1/−1 by opcode bit 0 (PUSH = −1, POP = +1).
- Branch & PC (Figure 6): a MUX selects the condition with opcode[1:0] (1 = JMP, eq = JEQ, gr = JGR, ls = JLS). DJNZ jumps when regD − 1 ≠ 0. PC is held during EXECUTE for STOP, GCD, FIB and SSUM.
- Flags (Figure 7): a comparator for accA and accB and the Prime ROM write FLAG_EQ, FLAG_GR and FLAG_LS in the EXECUTE state.
- ALU (Figure 8): adder, subtractor, multiplier, divider, logic, shift and rotate circuits run in parallel, and a 16-input MUX selects the result with opcode[3:0]. Rotation is done by concatenating accA with itself into 16 bits and then shifting.
- Hardware Loops (Figure 9): the red side generates the loop start and finish signals; the blue side holds the registers gcdA/B, fibN/fib0/fib1 and ssumPtr/ssumAcc/ssumDone.

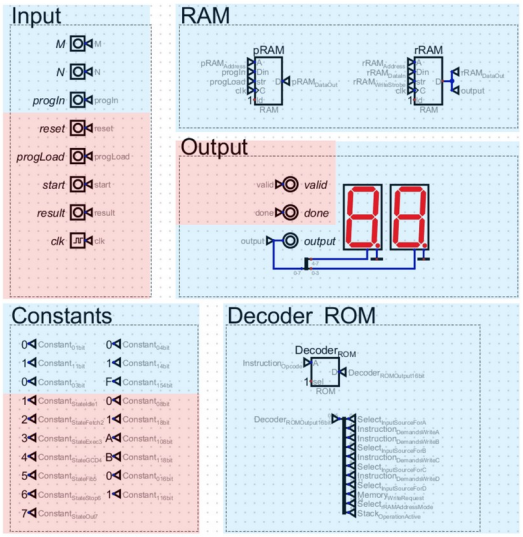


Figure 2 Input, RAM, Output, Decoder ROM

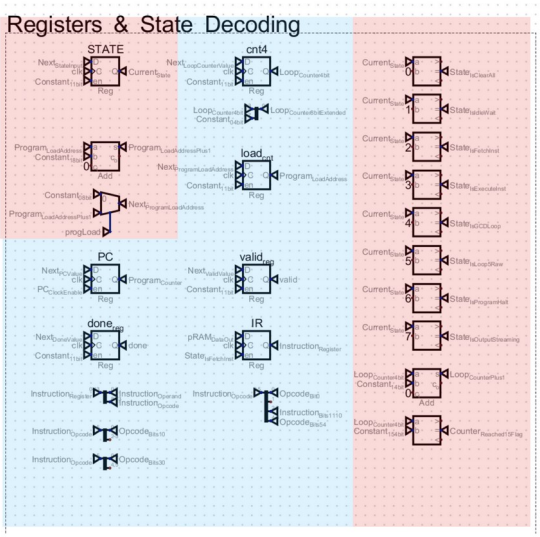


Figure 3 Registers & State Decoding

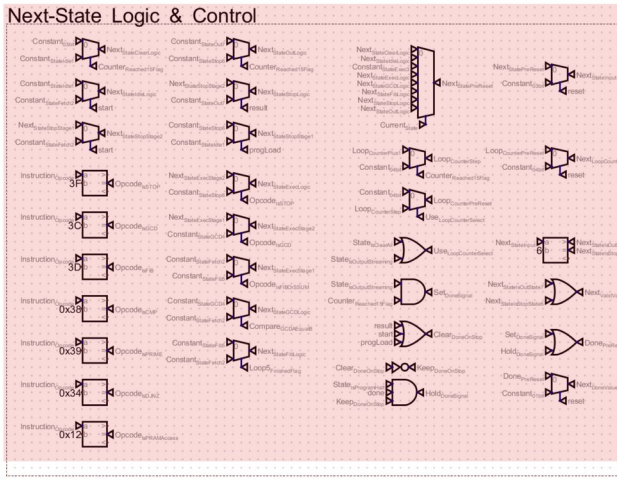


Figure 4 Next-State Logic & Control

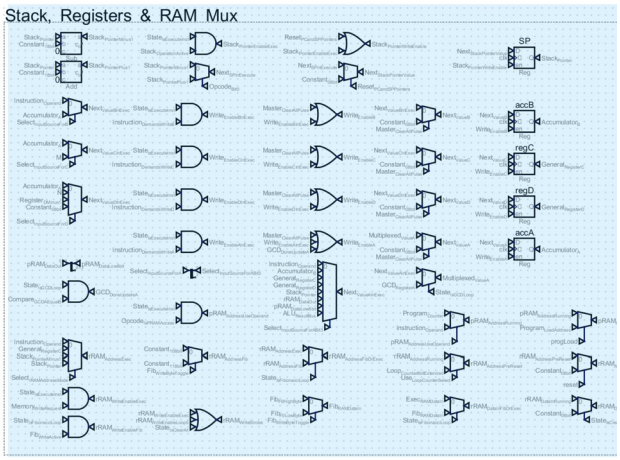


Figure 5 Stack, Registers & RAM Mux

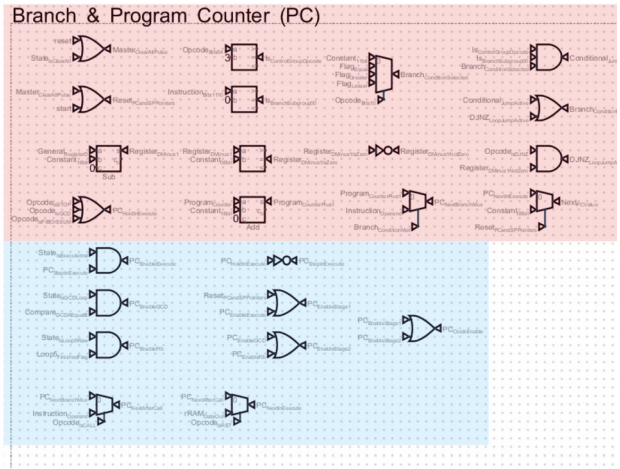


Figure 6 Branch & Program Counter

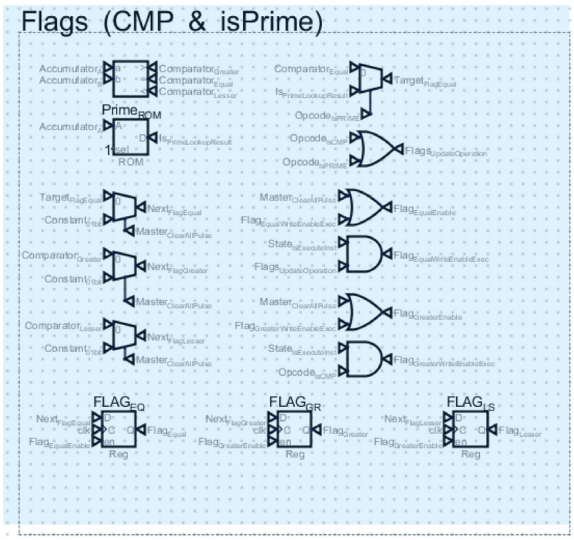


Figure 7 Flags (CMP & isPrime)

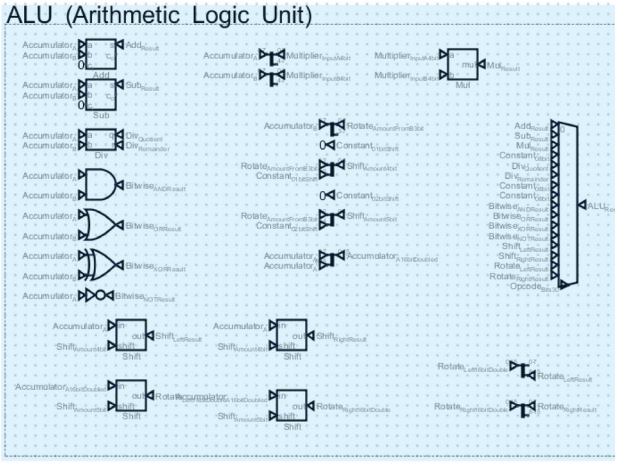


Figure 8 ALU

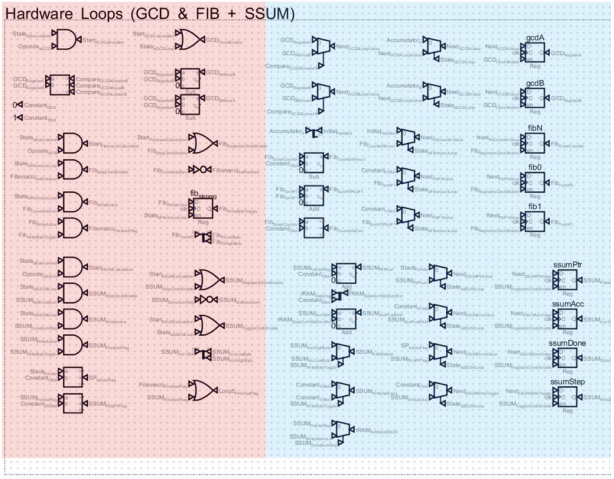


Figure 9 Hardware Loops

Chapter 6 Operation and Testing

6.1 Operating Procedure

1. Press reset and run the clock. The system clears itself and enters IDLE.
2. Set a 14-bit instruction on progIn, then press progLoad with one clock pulse. Repeat until the whole program is loaded, starting at address 0x00 and ending with STOP.
3. Set M and N (if needed), then press start. The system resets PC and SP and begins execution.
4. Wait until valid = 1, then press result. The system sends rRAM 0x00–0x0F out one value at a time until done = 1.

6.2 Test Plan

Table 5 Test items and expected results

Item	Method	Expected result	Result
Data / ALU	Load values and test every ALU instruction	accA matches the calculation	
Memory / Stack	Write/read rRAM; PUSH several values then POP	Values read back correctly; SP correct	
CALL / RET	Call a subroutine and return	PC returns to the instruction after CALL	
CMP, Jump, DJNZ	Compare then jump; set regD = 5 and loop	Jumps on the right condition; 5 iterations	
isPrime	accA is prime / not prime	eq = 1 and 0 respectively	
GCD	accA = 12, accB = 18	accA = 6	
FIB	accA = 10	rRAM[0x0A] = 0x00, [0x0B] = 0x37	
SSUM	PUSH 1, 2, 3 then SSUM	rRAM[0x0C] = 0x00, [0x0D] = 0x06	
STOP / result / done	Run until STOP, then press result	valid = 1; 16 values sent; done = 1	

Chapter 7 Conclusion

This project designed an 8-bit Mini CPU using the Digital simulator. It consists of registers accA, accB, regC, regD and SP, flags eq, gr and ls, a stack in rRAM, and an instruction set of four groups (42 basic instructions plus the additional CALL, RET and SSUM). The control unit is an 8-state FSM that uses a Decoder ROM to convert opcodes into control signals, and hardware loops allow GCD, FIB and SSUM to complete within a single instruction. Separating the Datapath from the Control Unit makes it easy to add new instructions in the remaining opcode slots.

References

[1] CEDT Final Project 2569, course 2110252 Digital Logic: Mini CPU + Stack (project specification). Chulalongkorn University.

[2] Wikipedia. Accumulator (computing). https://en.wikipedia.org/wiki/Accumulator_(computing)

[3] Neemann, H. Digital: A digital logic designer and circuit simulator. https://github.com/hneemann/Digital